

3618495 - Using OpenRewrite recipes to support SAP Commerce Cloud JDK21 Framework Update adoption

Component: CEC-SCC-PLA-PL (Customer Engagement and Commerce > SAP Commerce Cloud > Platform > Platform),
Version: 20, Released On: 16.01.2026

Symptom

This document guides you through the steps to use a tailored set of OpenRewrite recipes to reduce the manual efforts related to the SAP Commerce Cloud Framework Update. You will learn how to set up the local SAP Commerce Cloud project environment for OpenRewrite usage, run the recipes, and finalize your efforts related to SAP Commerce Cloud Framework Update. This procedure assumes you have a basic understanding of setting up SAP Commerce Cloud locally, and you already have a working local development setup based on the SAP Commerce Cloud commerce-suite package (preferably the latest version of 2211 for JDK17) with your customization included in the setup.

Although the shared set of recipes was pre-tested and used by SAP for internal Framework Update efforts, SAP cannot give any guarantees on the execution outcome on the customer code. Nevertheless, we encourage all customers to use and try this partially automated approach, which is expected to simplify and smoothen adjustments required for the Framework Update.

Manual adaptation to remaining changes and a regular update process are still required.

Limitations

There are also some limitations to be considered:

- OpenRewrite has very limited support for Groovy and JSP files. Most of the changes for these types of files must be adjusted manually, and you may see parsing errors related to those files while using the tool.
- Depending on the project size and number of configured extensions, execution of the OpenRewrite task can take from several minutes to even a few hours and require significant memory allocation for the Gradle process. The default proposed value for running the OpenRewrite tasks is 16GB of RAM (`-Dorg.gradle.jvmargs=-Xmx16G` parameter on the `./gradlew rewriteRun`). For large projects this value should be increased, for example to 32GB. If you run on Windows and use WSL to allocate additional memory, you might also need to adjust the `.wslconfig` file.
- Note that symbolic links (symlinks) are not supported during OpenRewrite Framework Update recipes processing. Ensure that all extensions configured for your SAP Commerce Cloud project are included as regular files.

Reason and Prerequisites

Before you begin, ensure the following:

- You have the SAP Commerce Cloud source code available locally (commerce-suite package and custom extensions).
- For Windows users: We strongly recommend using [Windows Subsystem for Linux \(WSL\)](#) for running Unix-style commands.
- You have an understanding of [identified upgrade steps to 2211-jdk21.1](#). Knowing the potential manual steps required before running OpenRewrite recipes can make the Framework Update adaptation process smoother.
- You familiarized yourself with both the recommended approach using the OpenRewrite Framework Update recipes and the manual option, and chose the best option for you. For more information, see [Framework Update - Java JDK 21 and Spring Framework 6.2](#).
- Available attachments: you made sure that you have the latest version of each available attachment. These attached files may have been updated since you last downloaded them. Last attachments update was on the **17th of November, 2025**.

Solution

Running below steps directly modifies files on the disk and doesn't produce a report of changes.

Step 1: Download Required Files

Download the following files available in the attachments section and place them in a directory accessible to your workspace:

- OpenRewrite recipes JAR:
sap-commerce-framework-update-recipes-1.1.3.jar
- Gradle initialization script:
init.gradle
- The jar file used for build.gradle generation:
yant.jar

Step 2: Prepare Your Local Environment

Use your current local environment for JDK17 (with the newest 2211 package, prior to 2211-jdk21).

Edit your local `config/localextensions.xml` to include:

- All extensions you want to update.
- Any required extension that is a dependency of the extensions you want to update, or the path to the location of dependencies.

If you want to update all your custom extensions at once, you could just use your current `localextensions.xml` setup.

Depending on your team's development flow, you should consider whether you want to update all your extensions at once or in predefined batches. In most cases, updating all extensions at once (if possible) should be simpler, but if your codebase is large and you have batches of extensions that can be tested individually, you should consider such an option as well.

Find below a sample configuration of local extensions that needs to be adjusted accordingly to your needs. Sample configuration assumes that you want to update: *myextension1*, *myextension2*, *myextension3* which depend on *myextension4* and *myextension5*, and all other required dependencies can be found in *HYBRIS_BIN_DIR* or a custom provided path.

This single example covers different setups of how your local environment could be structured, but a single real-life case should be simpler.

```
<hybrisconfig xmlns:xsi= 'http://www.w3.org/2001/XMLSchema-  
instance' xsi:noNamespaceSchemaLocation= '../bin/platform/resources/schemas/extensions.xsd' >  
  <extensions>  
    <!-- Example configuration of localextensions.xml for update. Order of extensions entries is not important.  
-->  
  
    <path dir= '${HYBRIS_BIN_DIR}' autoload= 'false' />  
    <!-- OPTIONAL CUSTOM PATHS WITH REQUIRED DEPENDENCIES FOR EXTENSIONS TO UPDATE-->  
    <path dir= '/<YOUR_PATH>/workspace/my-extensions' autoload= 'false' />  
  
    <!--EXTENSIONS TO UPDATE -->  
    <extension dir= "/<YOUR_PATH>/workspace/commerce-suite/hybris/bin/custom/myextension1" />  
    <extension dir= "/<YOUR_PATH>/workspace/my-extensions/myextension2" />  
    <extension name= "myextension3" />  
  
    <!-- EXTENSIONS DEPENDENCIES -->  
    <!-- If needed and not provided by the configuration of paths above -->  
    <extension name= "myextension4" />  
    <extension name= "myextension5" />  
  
  </extensions>  
</hybrisconfig>
```

Providing a minimal setup for the extensions that you want to update can reduce the memory usage and time required to run the recipes.

The `init.gradle` file is provided with a set of additional default exclusions configured to reduce the overall memory and resource consumption. The exclusions prevent extensions, that are part of the provided commerce-suite package and are located in `hybris/bin/modules` or `hybris/bin/platform` folders, from being added to the LST created by the OpenRewrite tool. Place your custom extensions in the `hybris/bin/custom` directory or outside the preconfigured exclusions. For more information about LST, see [Lossless Semantic Trees \(LST\)](#).

Step 3: Build and Initialize Your Project

At first, you are building your current version of SAP Commerce Cloud (at state before Framework Update), so you should still use JDK17 for the build process. Switch to JDK21 only after all the extensions used in your configuration are updated.

From your project's `hybris/bin/platform` directory, run:

```
./setantenv.sh  
ant clean all
```

You can also perform initialization of your project to ensure that the current configuration is valid.

```
ant initialize
```

These steps ensure your setup before the update can be compiled and your extension set is configured correctly.

Step 4: Generate and Adapt Gradle Configuration

OpenRewrite is used as a Gradle plugin, and a valid Gradle project configuration is needed. All presented commands in this step assume that your shell working directory is located in `hybris/bin/platform`.

Step 4.1: Use the new `yant.jar` library version:

The `yant.jar` file contains the resources required to run preconfigured Ant tasks for the Commerce project. In particular, for Framework Update, the Ant Gradle template generation task has been enhanced to create a complete list of Gradle project dependencies in the generated `build.gradle` file.

Since the exact SAP Commerce Cloud patch version on which the Framework Update is executed may vary, it's recommended to use the `yant.jar` file provided in this note when generating the Gradle project.

Procedure

1. Navigate to the `bootstrap/bin` subdirectory:

```
cd bootstrap/bin
```

2. Copy the `yant.jar` file downloaded from this note into the current directory:

```
mv <YOUR_DOWNLOAD_PATH>/yant.jar .
```

3. Return to the `hybris/bin/platform` directory:

```
cd ../..
```

Step 4.2: Generate Gradle build files

Run the following ANT task:

```
ant gradle
```

This will generate `build.gradle`, `settings.gradle`, `gradle.properties` in the `hybris/bin/platform` directory and `gradle.dependencies` files for each extension that is configured in your SAP Commerce Cloud local configuration. Verify that `build.gradle` file has been generated (it should be located in `hybris/bin/platform/build.gradle`).

Step 4.3: Generate Gradle Wrapper

Generated `build.gradle` supports Gradle in version 7.6.6. You may need to install this version on your machine. We suggest generating a Gradle Wrapper in the same directory in which the generated `build.gradle` is located. If you already have Gradle installed locally (in version 7), you can do this by running the following command:

```
gradle wrapper --gradle-version 7.6.6
```

Step 4.4: Verify if the project compiles using Gradle

Verify if the project is building by running the following command (OpenRewrite requires a project to compile):

```
./gradlew assemble testClasses
```

Step 5: Set up Environment Variables

In the shell that will be used for running the OpenRewrite command, set the environment variable `REWRITE_JAR_PATH` to point to the location of the downloaded JAR file with OpenRewrite recipes. This environment variable is required by the `init.gradle` script while running OpenRewrite tasks. For example:

```
export REWRITE_JAR_PATH= "/<YOUR_PATH>/sap-commerce-framework-update-recipes-1.1.3.jar"
```

Or when using WSL:

```
export REWRITE_JAR_PATH= "/mnt/c/<YOUR_PATH>/sap-commerce-framework-update-recipes-1.1.3.jar"
```

Step 6: Temporarily Move Git Repository Files

Determine whether your SAP Commerce Cloud project is part of a Git repository (if `.git` folder is present anywhere on the directory path of the project).

- **If it isn't**, no action is needed — relative paths for code migration will be calculated as expected.
- **If it is**, move to the root directory of the git project and rename the `.git` directory to `.gitTemporary`.

```
mv .git .gitTemporary
```

This prevents OpenRewrite from detecting the repository and altering the relative path structure which is expected during migration. The change should be reverted after the OpenRewrite migration is executed.

Step 7: Run the Recipes

Keep in mind that this process is memory-intensive and time-consuming. Depending on the size of your project, it might take even several hours. For large projects, you may need to adjust the `-Dorg.gradle.jvmargs` setting to higher values. Pay also special attention to the proper setting of `$COMMERCE_UPDATE_PATHS` variable so it includes only the sources you want to update.

You can also leverage the `rewriteDryRun` command that performs a dry-run update and instead of applying changes on source files, it creates a report (patch file) with changes that would be applied. Keep in mind that time of execution will be very close to the time of the regular `rewriteRun` task execution. To learn more, see [The "dryRun" task](#).

To apply the recipes to your extensions, run the following commands:

ulimit -n 81920 #increase the limit of concurrently opened files

```
./gradlew rewriteRun \
--init-script=$PATH_TO_INIT_GRADLE \
-PcommerceUpdatePaths=$COMMERCE_UPDATE_PATHS \
--no-daemon \
-Dorg.gradle.jvmargs=-Xmx16G \
-Drewrite.activeRecipe=com.sap.cx.rewrite.java.SelectiveCommerceCloudFrameworkUpdate
```

Parameters:

\$PATH_TO_INIT_GRADLE	Path to your downloaded <code>init.gradle</code> .
\$COMMERCE_UPDATE_PATHS	The set of paths separated by (pipe) and relative to the <i>hybris/bin/platform</i> directory which match extensions that should be updated. Detailed example can be found in the section below. If you provide the <code>\$COMMERCE_UPDATE_PATHS</code> value directly in the Gradle command (not as an environment variable), remember to embed its value in quotes: <code>-PcommerceUpdatePaths='./custom/myextension1'</code> When running the Gradle task, the value of the parameter will be printed at the beginning of the log.

How to Specify commerceUpdatePaths

You can categorize custom extensions for the local project into two groups:

- Extensions that you want to update and use OpenRewrite on. Usually, these are customer extensions.
 - If you want to run OpenRewrite recipes in batches, then you will also have your remaining custom extensions that you don't want to use OpenRewrite on at this moment. For each batch you need to use the not-yet-updated version of them, so the project can compile to run the OpenRewrite.
- Extensions delivered with the SAP Commerce Cloud Packages. These extensions were already updated by SAP and are available in new package. For updating customer extensions you should use the not-yet-updated SAP Commerce Cloud Packages.

As you want to run the recipes only on the first group, you can pass additional information to the OpenRewrite Gradle task by specifying the `-PcommerceUpdatePaths` Gradle parameter. The parameter is interpreted by one of the recipes and treated as a precondition for applying the next recipes.

The parameter value should be a set of paths separated by | (pipe) and relative to the `hybris/bin/platform` directory (the location from where the OpenRewrite Gradle task is run) which match the extensions that should be updated.

Example	Sample Directory Structure
Take a look at a sample directory structure: We want to update <i>myextension1</i> and <i>myextension3</i> which are located in <code>hybris/bin/custom</code> and <i>myextension2</i> which is located outside the commerce-suite package. Taking into consideration the project structure from the graphics on the right and our requirement to update only those three extensions, we should prepare following paths: <pre>COMMERCE_UPDATE_PATHS="./custom/myextension1 ../../../../../my-extensions/myextension2 ../custom/myextension3"</pre>	<pre>__workspace __my-extensions __myextension2 ← Extension to run OpenRewrite on __myextension5 ← Extension for which recipes won't be applied __commerce-suite __hybris __bin __modules __platform __build.gradle ← OpenRewrite execution path to which paths are relative __custom __myextension1 ← Extension to run OpenRewrite on __myextension3 ← Extension to run OpenRewrite on __myextension4 ← Extension for which recipes won't be applied __config __localextensions.xml</pre>

If you also want to update extensions *myextension4* and *myextension5*, the parameter value could look like this:

```
COMMERCE_UPDATE_PATHS="./custom/myextension1|../../../../../my-extensions/myextension2|../custom/myextension3|../custom/myextension4|../../../../../my-extensions/myextension5"
```

or simpler, if all extensions from the `my-extensions` and `custom` directories should be updated, the following parameter value could be used:

```
COMMERCE_UPDATE_PATHS="../../custom/|../../../../../my-extensions/"
```

Bear in mind that `platform` and `modules` directories are always excluded from the update to limit resources consumption.

If running the recipes doesn't result in any changes to one or more of the extensions, double-check that the provided `commerceUpdatePaths` parameter is correct. For more information, see point 2 of the Troubleshooting section at the end of this documentation.

If in step 6 `.git` folder was renamed, move to the root directory of the git project and revert the changes by running:

```
mv .gitTemporary .git
```

Step 8: Switch to JDK 21

After updating your extensions and all dependencies, **setup your local JDK21 environment by using SAP Commerce Cloud 2211.jdk21 package and updated custom extensions and change your Java version to JDK21**. You should use [sapmachine jdk21](#) (download [link](#)). If your code contains any property files or settings that reference an older Java version, be sure to update those as well.

To quickly switch between the Java versions for the workspace, you can use the [jenv](#) tool or [sdkman](#).

Bear in mind that OpenRewrite recipes doesn't cover all necessary changes to update SAP Commerce Cloud to JDK 21 and your updated project most likely won't compile with JDK21 out of the box requiring further adaptation (see step 9).

Step 9: (Optional) Generate Gradle Build Files

If you use the Gradle project structure to import locally SAP Commerce Cloud into your IDE, you should regenerate the new Gradle configuration adjusted to the updated Gradle 8.8 version.

Generate Gradle build files:

```
. ./setantenv.sh
ant gradle
```

Update the Gradle Wrapper version which will be used for the commerce-suite (you might need to switch the used Gradle version in your IDE configuration as well):

```
gradle wrapper --gradle-version 8.8
```

Step 10: Attempt to Build and Identify Required Manual Updates

First, try to build and initialize SAP Commerce Cloud to reveal most obvious compilation issues.

Run the following commands:

```
. ./setantenv.sh
ant clean all
ant initialize
./hybrisserver.sh
```

If the build fails, consult the [list of manual update steps](#) for common issues and solutions that frequently occur when adjusting to Framework Update changes. Resolve any issues as needed.

Step 11: Proceed with your regular update process

After confirming the updated extensions are compiling and the application server is running without issues, continue with your regular update process.

Troubleshooting

1. Too many open files error during the OpenRewrite task execution.

Problem

If execution of `./gradlew rewriteRun` ends with:

Problem in daemon expiration check

```
org.gradle.api.UncheckedIOException: java.io.IOException: Too many open files at
org.gradle.internal.UncheckedException.throwAsUncheckedException(UncheckedException.java:62) at
org.gradle.internal.UncheckedException.throwAsUncheckedException(UncheckedException.java:41) at
```

```

org.gradle.cache.internal.DefaultFileLockManager.lock(DefaultFileLockManager.java:113) at
org.gradle.cache.internal.DefaultFileLockManager.lock(DefaultFileLockManager.java:96) at
org.gradle.cache.internal.DefaultFileLockManager.lock(DefaultFileLockManager.java:91) at
org.gradle.cache.internal.OnDemandFileAccess.readFile(OnDemandFileAccess.java:41) at
org.gradle.cache.internal.SimpleStateCache.get(SimpleStateCache.java:49) at
org.gradle.cache.internal.FileIntegrityViolationSuppressingPersistentStateCacheDecorator.get(FileIntegrityViolationSu
at
org.gradle.launcher.daemon.registry.PersistentDaemonRegistry.getAll(PersistentDaemonRegistry.java:71) at
org.gradle.launcher.daemon.registry.PersistentDaemonRegistry.getDaemonsMatching(PersistentDaemonRegistry.java:116)
at
org.gradle.launcher.daemon.registry.PersistentDaemonRegistry.getIdle(PersistentDaemonRegistry.java:84) at
org.gradle.launcher.daemon.server.CompatibleDaemonExpirationStrategy.checkExpiration(CompatibleDaemonExpirationStrate
at
org.gradle.launcher.daemon.server.expiry.AllDaemonExpirationStrategy.checkExpiration(AllDaemonExpirationStrategy.java
at
org.gradle.launcher.daemon.server.expiry.AnyDaemonExpirationStrategy.checkExpiration(AnyDaemonExpirationStrategy.java
at
org.gradle.launcher.daemon.server.MasterExpirationStrategy.checkExpiration(MasterExpirationStrategy.java:73) at
org.gradle.launcher.daemon.server.Daemon$DaemonExpirationPeriodicCheck.run(Daemon.java:268) at
org.gradle.internal.concurrent.ExecutorPolicy$CatchAndRecordFailures.onExecute(ExecutorPolicy.java:64) at
org.gradle.internal.concurrent.ManagedExecutorImpl$1.run(ManagedExecutorImpl.java:48)

```

Solution:

- Verify if JDK17 you are using is the latest SAP Machine >= 17.0.15 by running `java -version`. Some older JDK versions (17.0.9) had issues with honoring the customization of `ulimit -n 81920` setting.
- Check if you specified the `ulimit` correctly by executing: `ulimit -n` in the same shell as the OpenRewrite Gradle command.
- If the problem persists, try to increase the `ulimit -n` value.

2. No or limited results of running recipes.

Problem:

After the OpenRewrite `rewriteRun` process ends, you notice that no or a limited number of changes were created for your extensions, for example, you notice that removed `javax.servlet.*` packages were not automatically changed to `jakarta.servlet.*` packages.

Solution:

Double-check if the `-PcommerceUpdatePaths` parameter value was specified correctly. Despite the process is running for a long time if the value is specified incorrectly the changes are not applied. OpenRewrite parses full project structure to create its LST to allow code modifications. This is a memory-intensive and time-consuming task. Only after this initial work is done the custom `-PcommerceUpdatePaths` setting limits the paths for which the changes are applied. If you specify the `-PcommerceUpdatePaths` incorrectly then no or not all changes are applied, but the execution process already had to perform the resource/time heavy operations. OpenRewrite doesn't allow for persisting the LST, so with each `rewriteRun` invocation, these values need to be calculated once more. It's important to set the value correctly to have a valid result.

You can also read the beginning of the log and find some useful information.

- Extensions that match at least one of the paths. Have a look at this example output:

```

The following extensions match update pattern:
- myextension1
- myextension2
- myextension3

```

- Each commerce update path without a matching extension will trigger a warning. Here is a sample output:
WARNING: The following patterns do not match any extension:
WARNING: - '../myextension1'
- In case no extension was matched, the task execution is stopped with an error. This is an output you'll see in this scenario:
* What went wrong:
The update pattern does not match any extension!

If you still have doubts why your path is incorrect, you can find extensions paths relative to the `hybris/bin/platform` defined in the `hybris/bin/platform/settings.gradle` file and compare them with your path:

```

include(':myextension1')
project(':myextension1').projectDir = file('../../../my-extensions/myextension1')

```

3. Out of memory error when running OpenRewrite task.

Problem:

When running the OpenRewrite `rewriteRun` Gradle task it ends with OOM (out of memory) error.

Solution:

- Go through your `localextensions.xml` file and exclude any of yours extensions which isn't required for extensions planned to update to work (extensions located in `platform` and `modules` are already excluded by default). Although these extensions aren't updated as they are a part of the project structure, they are loaded and parsed by OpenRewrite which puts pressure on memory consumption. Remember to run `ant gradle` task after modifying the `localextensions.xml` file to reflect configuration changes to Gradle project.
- Increase the `-Dorg.gradle.jvmargs` parameter value, for example to `-Xmx32G`
- If you don't have more memory available, you could try running the OpenRewrite several times on independent batches of your extensions. Identify groups of extensions which does not need each other to work, configure your `localextensions.xml` accordingly and run the recipes for each group separately.

4. Project fails to compile using `./gradlew assemble` but successfully compiles using `ant all`.

Problem:

In step 4.4 you encounter a failure related to compiling the project using the `./gradlew assemble` task, despite the fact that the execution of `ant all` task in previous steps finished successfully. The compilation errors indicate missing classes – either in import statements or within the extension code itself – that are defined in this or another extension.

There are two common possible reasons behind that:

a) Missing dependency between extensions

When building with `ant`, such missing dependency declarations are often tolerated, as long as both extensions are included in the project configuration, and the dependent extension is loaded first. Gradle, however, enforces stricter dependency management.

To avoid build failures, ensure that all required extensions are explicitly declared in the `extensioninfo.xml` file. Without this, the `ant gradle` task won't generate the necessary Gradle project dependencies, leading to compilation errors.

Solution:

- Add missing dependencies to your `extensioninfo.xml` configuration.
- Regenerate the Gradle project configuration using `ant gradle` task.
- Try to build the project using `./gradlew assemble` again.

For more information, see [Extension Dependencies](#).

b) Code in source directory (src) depends on code in test source directory (testsrc)

You can find the missing classes by analyzing the compilation issue and checking if the class that is missing in the import is located in the test code (`testsrc` directory) and is used by the class defined in the source code (`src` directory).

Ant-based build is less restrictive in this regard. Gradle, on the other hand, enforces a stricter separation between the `testsrc` and `src` directories.

Solution:

The appropriate approach to resolving this issue is to **refactor the application code** to eliminate the usage of the test classes in the source code. Following best practices, test code should not be part of the application code.

However, as a temporary solution to continue utilising OpenRewrite recipes for the SAP Commerce Cloud Framework Update, you can follow these steps:

- Copy the missing classes from the `testsrc` into the `src` folder of **your** extension with the same package.
- Try to build the project using `./gradlew assemble` again (do not call `ant clean all` again since `ant` complains about the duplicate classes).
- After the update, restore the previous state and consider a permanent fix as described above.

5. Recipe execution fails due to error related to git repository presence.

Problem:

The `rewriteRun` Gradle task fails with an error message indicating that OpenRewrite automatically selected a Git repository to override the default base directory. This behavior conflicts with the file selection mechanism expected when using the `commerceUpdatePaths` parameter. As a result, the recipe execution was terminated without generating any results.

Solution:

You should follow the guidance in step 6 (Temporarily Move Git Repository Files).

6. Recipe execution fails due to error related to specific code constructs in the customer implementation.

Problem:

The `rewriteRun` Gradle task fails with an error message indicating an unexpected issue during recipe execution, with the stack trace pointing to customer code. As a result, the execution was aborted without generating any results.

Solution:

Although the set of recipes was pre-tested and successfully used during the internal Framework Update effort, certain specific code constructs (for example, chained method invocations or uncommon patterns) may still cause some recipes to fail unexpectedly.

In such cases, we recommend temporarily commenting out the problematic code fragment during the migration process and restoring it—with any necessary adjustments—after recipe run is complete.

While the Framework Update OpenRewrite recipes are not officially supported, you may contact your SAP representative through email to help us improve the tooling in future releases. Include the full execution log and the relevant code construct that caused the failure.

7. Generation of Gradle Wrapper fails.

Problem:

When running the `gradle wrapper --gradle-version 7.6.6` command, the process ends with an error and the wrapper configuration isn't generated.

As the project configuration differs depending on your extension set, you may encounter an error similar to that shown below but located in other places of the generated `build.gradle` file.

```
* What went wrong:
A problem occurred evaluating root project 'platform'.
> Could not get unknown property 'outputDir' for main Java source of type org.gradle.api.internal.file.DefaultSourceDirectorySet.
```

Solution:

The generated `build.gradle` is compatible with Gradle in version 7.6.6. Some of the APIs used in the build script were changed in following major versions of Gradle.

To overcome the issue, make sure that the Gradle installation you use for running the wrapper task is in version 7.X (preferably 7.6.6). If not, install Gradle in this version and rerun the wrapper task to generate configuration.

8. Your custom extension utilizes Lombok and doesn't compile with Gradle.

Problem:

Your project uses Project Lombok within custom SAP Commerce Cloud extensions and the extension fails to compile with Gradle.

Solution:

SAP Commerce Cloud doesn't natively support the usage of Project Lombok. Therefore, the generated Gradle files don't contain configuration for the Lombok library. To make the Gradle build function correctly with Lombok, you need to manually adjust the content of the generated `build.gradle`.

Find the names of the Gradle subprojects that need adjustments based on the error log from the Gradle task execution, or check them in the generated `build.gradle` file. Gradle project names are based on the extension names.

Insert an additional configuration block into the generated `build.gradle` with the required configuration for Project Lombok. The new configuration block should be placed on the top level below the subprojects section, as this section must be processed first. Use the example below as the basis for your changes:

```
ext {
    lombokVersion = '1.18.32' // Example version, us the same as configured in your extensions
}

// List the Gradle subproject to which you want to apply the configuration
def lombokProjects = [':myCustomExtension', ':myCustomExtension_web', ':myCustomExtension2', ':myCustomExtension2_web', ':myCustomExtension2_addon']

configure(lombokProjects.collect { project(it) }) {
    dependencies {
        compileOnly "org.projectlombok:lombok:${lombokVersion}"
        annotationProcessor "org.projectlombok:lombok:${lombokVersion}"
    }

    // Optional: if you're using tests that also use Lombok
    // testCompileOnly "org.projectlombok:lombok:${lombokVersion}"
    // testAnnotationProcessor "org.projectlombok:lombok:${lombokVersion}"
}
```

After applying the changes, recompile the project using Gradle. Remember that if you regenerate the build configuration, you need to reapply the changes.

Attachments

File Name	File Size	Mime Type
init.gradle	19	application/octet-stream
sap-commerce-framework-update-recipes-1.1.3.jar	223	application/java-archive

